

Roadmap for CP

- By C3

Topics to study:

(Note: this is the whole CP syllabus, we don't expect that you learn all the topics for Recruitment. Go step by step, and cover whatever you can, focus on **quality over quantity.**)

1. Basics
 - C++ basics (variables, if-else, loops, functions, recursion, arrays, pointers)
 - Time and Space complexity (Big-O notation)
2. STL library
 - vectors, sets, multisets, maps, pairs
 - Iterators
 - Basic library functions, custom comparators (for sort)
 - Struct
3. Prefix Array
 - Solving basic ranged query type problems using prefix/suffix array precomputation.
 - Difference array technique
 - 2D prefix array
4. Binary Search (**V. Imp**)
 - BS with integer range
 - BS with floating point values
 - BS on answer technique
 - Ternary Search (less imp)
5. 2 pointer and sliding window technique.
6. Number theory
 - Modulus properties
 - Binary exponentiation
 - Euclidean Algorithm for GCD
 - All possible factors in $O(\sqrt{n})$
 - Sieve of Eratosthenes (**V. Imp**)
 - Smallest Prime factor (SPF), Prime factorization in $O(\log n)$ per query
 - Modulo inverse
 - ETF, chinese remainder theorem, and other theorems.. (**less imp.**)

7. Bit manipulation

- Bitwise operators(&, |, ^, !, >>, <<) and their properties.
- Basic techniques like, check a bit, set/unset a bit, flip a bit.
- Bitmasking

8. Stacks, queues, priority queues

- Next/prev greater value using stack (Monotonic stack)

9. Dynamic Programming

- Top-Down approach (memoization)
- Bottom-Up approach (Tabulation)
- Knapsack problems
- Longest common subsequence(LCA), matrix chain multiplication (MCM) problems.
- DP with bitmasking
- Digit-DP
- **Must practise CSES DP problemset.**

10. Combinatorics & Probability

- nCr calculations
- Inclusions/exclusion principle
- Probability & expected value

11. Graphs & Trees

- DFS/BFS traversal
- Shortest path techniques (BFS, 0-1 BFS, dijkstra, belmon ford, floyd warshall)
- DSU, MST
- DP on trees, Diameter of Tree
- SCC
- Binary lifting, LCA
- Rerooting technique on trees

12. String algos

- String hashing
- Rabin Karp, KMP, Z-function (knowing any 1 is ok)
- Manacher algorithm

13. Tries

- Knowing the Trie data structure, and basic problems related to string prefixes
- Min-Xor/Max-Xor problems

14. Ranged Queries

- Prefix/difference array technique (already known)
- Sparse table
- Fenwick Tree
- Segment Tree

- Segment Tree with lazy propagation

How to Proceed:

1. Start studying the topics(resources given below), and keep practising problems alongside.
2. You can start giving contests after learning the basics (1). Platforms mentioned below. Just be consistent and have patience, you will see improvements.
3. When you study a topic, practise a lot of problems on that topic, before proceeding to the next topic.
4. When stuck on a problem, try to debug yourself first, if you are unable to, look at the editorials. If still stuck, ask your doubts in the C3 discord server.

From where to study the topics?

(U can follow **any one**)

1. Offline CodeISM classes
2. Chirag Jain sir's playlist:
 - Level 1:
<https://youtube.com/playlist?list=PL40a3hTWsqXAw6cnRALe06QzPfeqnOBo6&si=DA50CmFeCad6Xe4h>
 - Level 2:
<https://youtube.com/playlist?list=PL40a3hTWsqXBFAKwLv-02tsKOLCud6hvf&si=2CnsToIDv7T3I3VM>
3. Search about the topics and read from cp-algorithms.com or GFG

Note: You can follow any other lectures of choice to study the topics, as long as you are able to understand its fine. But, make sure you are not following a lengthy playlist and wasting time on only learning DSA-theory (topics not mentioned above need not be studied now), so that you can give more time to **Problem-solving (which is the most important for CP)**.

Coding Platforms:

Make Accounts here

(Take a good username)

1. <https://codeforces.com/>
2. <https://atcoder.jp/home>
3. <https://www.codechef.com/>
4. <https://www.hackerrank.com/auth/login>
5. <https://cses.fi/register/>
6. <https://www.spoj.com/>

Compiler/Language to use on these platforms for C++ (make sure to select before submission)

- Codeforces: GNU G++17 / GNU G++20
- Atcoder: C++ (GCC 9.2.1)
- Codechef: C++17
- Hackerrank: C++14
- CSES: C++
- Spoj: C++14 (gcc 8.3)

Where to give contests?

For beginners:

1. Codechef starters (every Wed)
2. Atcoder ABC contest (every Sat)
3. CF div 3, CF div 4

Once you are little comfortable with above:

1. Codeforces (all Divs)
2. Codechef starters

How to give contests?

1. Be consistent and don't miss any contests.
2. **Upsolve at least 1(or 2) problems** after the contest - Try again by yourself first, if still unable to think, then look at the solution. If it involves a topic, which is not yet studied by you, don't worry.. just bookmark that problem, and come leave it for now. (Come back to it after you study that topic)
3. Analyse the contest, and note down your silly mistakes (if any) during the contest, so that u don't repeat it again

How to practise Problems?

1. Topic-Wise practise

- You will be given some practice problems in the C3 group, attempt those.
- For basics (if you are starting) - practise on Hackerrank :
<https://www.hackerrank.com/domains/algorithms>
(Apply **warm-up**, **Implementation**, filter)
- CSES problemset : <https://cses.fi/problemset/>
- Codeforces Edu : <https://codeforces.com/edu/courses>
(Go to pilot courses -> enroll)

2. Rating-wise Problem practise

- CP31 sheet : <https://www.tle-eliminators.com/cp-sheet>

- Codechef practise problems (filter based on difficulty) :
<https://www.codechef.com/practice-old>
- Codeforces Problemset (filter based on difficulty) :
<https://codeforces.com/problemset>

Note: both topic-wise and rating-wise practise is needed. When you study a topic, practise some topic-wise problems to strengthen that topic. Alongside, keep practising random rating-wise problems in general.